EXPERIMENT : 19

AIM:

Experiment to understand Logistic Regression for a given data set.

DESCRIPTION:

Understand the Logistic Regression algorithm for the dataset given.

CODE:

```
import numpy as np
import pandas as pd

# Load dataset
df = pd.read_csv('/content/social.csv')

# Display dataset
print(df.head())

# Select features and label
features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values

print(features)
print(label)

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Find a random state where test score is greater than train score
for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )

    model = LogisticRegression()
    model.fit(x_train, y_train)

    train_score = model.score(x_train, y_train)
```

```
    test_score = model.score(x_test, y_test)

    if test_score > train_score:
        print("Test {} Train {} Random State {}".format(
            test_score, train_score, i
        ))

# Split the dataset using a fixed random state
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)

# Train the final model
finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)

# Display accuracy
print("Training Accuracy:", finalModel.score(x_train, y_train))
print("Testing Accuracy:", finalModel.score(x_test, y_test))

# Classification report for test data
from sklearn.metrics import classification_report

y_pred = finalModel.predict(x_test)

print(classification_report(y_test, y_pred))
```

OUTPUT:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000],
       [    48,  29000],
       [    45,  22000],
       [    47,  49000],
       [    48,  41000],
       [    45,  22000],
       [    46,  23000],
       [    47,  20000],
       [    49,  28000],
       [    47,  30000],
       [    29,  43000]
```

```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1])
```

```
Test 0.8625 Train 0.85 Random State 5
Test 0.8625 Train 0.859375 Random State 6
Test 0.8875 Train 0.8375 Random State 7
Test 0.8625 Train 0.8375 Random State 9
Test 0.9 Train 0.840625 Random State 10
Test 0.8625 Train 0.85625 Random State 14
Test 0.85 Train 0.84375 Random State 15
Test 0.8625 Train 0.85625 Random State 16
Test 0.875 Train 0.834375 Random State 18
Test 0.85 Train 0.84375 Random State 19
Test 0.875 Train 0.84375 Random State 20
Test 0.8625 Train 0.834375 Random State 21
Test 0.875 Train 0.840625 Random State 22
Test 0.875 Train 0.840625 Random State 24
Test 0.85 Train 0.834375 Random State 26
```

```
Training Accuracy: 0.81875
Testing Accuracy: 0.9625
Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.98      0.97        55
           1       0.96      0.92      0.94        25

    accuracy                           0.96        80
   macro avg       0.96      0.95      0.96        80
weighted avg       0.96      0.96      0.96        80
```

RESULT:

The Logistic Regression algorithm was successfully implemented on the given dataset. The model was used to classify the data into appropriate categories and predict the output based on the input features.